

ICS Lab0 实验报告

任务 1：Git 基础问题

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我曾在一次小组编程作业中与同学合作。我们按功能划分任务，分别负责输入处理、核心逻辑和测试，通过聊天软件交换代码，最后由一名同学统一整合。这种方式比较直接，但容易出现文件版本混乱和修改相互覆盖的问题，也让我意识到使用版本控制工具的重要性。

2. Git 为什么要设计“暂存—提交”两个步骤？

暂存区允许开发者从工作区的修改中选择本次要提交的内容，而不必一次提交所有修改。例如，同时修改了程序功能和说明文档时，可以先暂存并提交功能修改，再单独提交文档修改。

暂存是组织和检查本次修改的过程，提交则是将暂存区的内容保存为一个版本。这样可以让每次提交围绕一个明确的目的，便于查看历史、定位问题和回退版本。

3. `git branch` 和 `git branch -a` 的区别是什么？

`git branch` 只列出本地分支，并用 `*` 标记当前所在分支。本次实验创建了 `main` 和 `feature` 两个本地分支。

`git branch -a` 同时列出本地分支和远程跟踪分支，例如 `remotes/origin/main`。远程跟踪分支是本地保存的远程分支状态记录，该命令不会实时查询 GitHub；需要通过 `git fetch` 更新相关记录。

任务 2：创建仓库、修改代码并提交

1. 从模板创建并克隆个人仓库

在课程模板仓库页面选择 **Use this template → Create a new repository**，创建个人仓库 `nannanpiano/ICS_Lab0`。随后在 Mac 终端执行：

```
cd /Users/nanny/Project/course/ics
git clone https://github.com/nannanpiano/ICS_Lab0.git
cd ICS_Lab0
git status
```

克隆后位于 `main` 分支，本地分支与 `origin/main` 一致，工作区干净。之后在 VS Code 中打开仓库文件夹，编辑 `main.c`。

2. 修改 TODO 并创建提交

将 `main.c` 中原来的输出语句：

```
printf("Hello, world!\n");
```

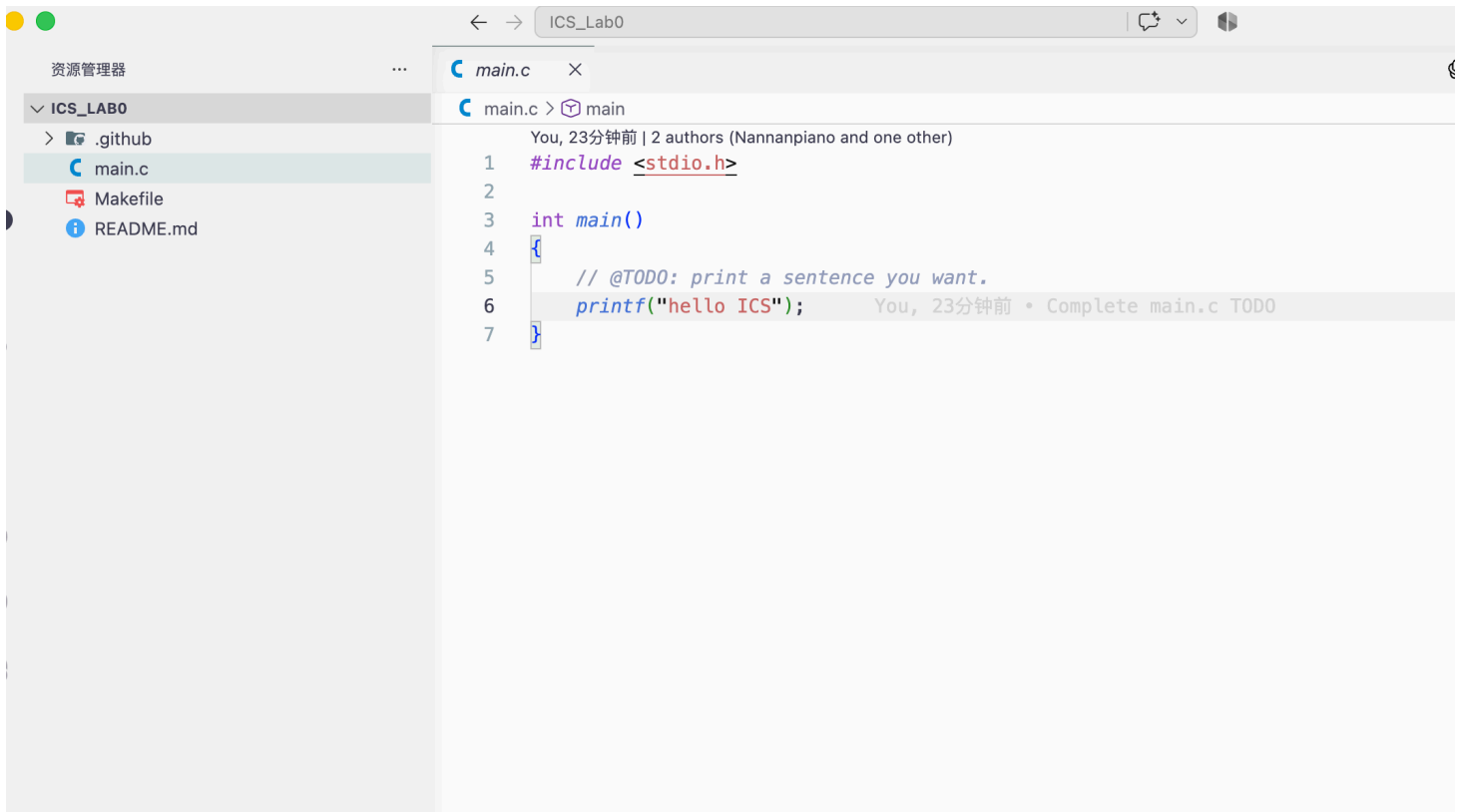
修改为：

```
printf("hello ICS");
```

保存文件后，在终端暂存并提交修改：

```
git add main.c
git commit -m "Complete main.c TODO"
git status
```

生成的提交编号为 `c3e52ef`，提交结果显示 `1 file changed, 1 insertion(+), 1 deletion(-)`，说明对 `main.c` 的修改已保存到本地版本历史中。随后执行 `git push`，将提交上传至 GitHub。



```
[nanny@ZenandeMacBook-Pro ICS_Lab0 % git push
枚举对象中：5，完成。
对象计数中：100% (5/5)，完成。
使用 10 个线程进行压缩
压缩对象中：100% (3/3)，完成。
写入对象中：100% (3/3)，299 字节 | 299.00 KiB/s，完成。
总共 3 (差异 2)，复用 0 (差异 0)，包复用 0 (来自 0 个包)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/nannanpiano/ICS_Lab0.git
    adedbb9..c3e52ef  main -> main
[nanny@ZenandeMacBook-Pro ICS_Lab0 % git status
位于分支 main
您的分支与上游分支 'origin/main' 一致。

未跟踪的文件：
  (使用 "git add <文件>..." 以包含要提交的内容)
    .DS_Store

提交为空，但是存在尚未跟踪的文件（使用 "git add" 建立跟踪）
nanny@ZenandeMacBook-Pro ICS_Lab0 %
```

任务 3：阅读概括与学习体会

1. 《Commit Message 规范》内容概括

文章介绍了如何用统一格式编写提交说明。文中采用的 Angular 规范将提交说明分为标题、正文和页脚，其中标题必填。标题格式为 `<type>(<scope>): <subject>`，依次表示修改类型、影响范围和简短描述，范围可以省略。例如，`feat` 表示新增功能，`fix` 表示修复问题，`docs` 表示文档修改。正文解释修改原因，页脚可说明不兼容变更或关联问题。规范的提交说明有助于检索历史、理解修改目的，并配合工具生成更新日志。这是团队可以采用的约定，并非 Git 本身强制要求的格式。

2. 《语义化版本》内容概括

语义化版本使用 `主版本号.次版本号.修订号` 传达软件更新的性质。对于已有稳定公共接口的软件，不兼容的接口修改增加主版本号；兼容旧版本的功能新增增加次版本号；兼容旧版本的问题修复增加修订号。例如，从 `1.2.3` 出发，修复问题可发布为 `1.2.4`，新增兼容功能可发布为 `1.3.0`，不兼容的接口变化则发布为 `2.0.0`。已经发布的版本不能直接更改内容，而应发布新版本。这套规则帮助使用者判断升级兼容性，管理软件依赖。

3. 为什么要学习 Git?

我认为，学习 Git 可以让代码修改有记录、可比较、可追溯。每次提交都保存一个版本，出现问题时可以检查修改历史，并恢复到合适的状态，避免依靠多个备份文件管理代码。

Git 也支持并行开发。在本次实验中，我在 `main` 和 `feature` 分支上分别修改了同一行代码，合并时产生冲突。通过检查双方的内容、手动修改并提交，我理解了分支能够隔离不同的开发工作，而冲突处理需要开发者判断最终保留的内容。

结合阅读材料，我认识到 Git 记录修改历史，清晰的提交说明解释修改目的，语义化版本则向使用者传达发布版本之间的兼容关系。掌握这些方法，既有助于个人维护项目，也能减少团队协作中的沟通成本。

任务 4：分支管理与合并冲突处理

1. 创建 feature 分支并提交修改

在任务 2 的提交 `c3e52ef` 基础上，创建并切换到 `feature` 分支：

```
git status
git switch -c feature
git branch
```

此时 `git branch` 输出中的 `* feature` 表明当前位于新分支。在 VS Code 中将输出语句改为：

```
printf("hello from feature");
```

保存后执行：

```
git diff -- main.c
git add main.c
git commit -m "Change greeting on feature branch"
```

生成提交 `f9dc407` 。

2. 切回 main 分支并修改同一行

执行：

```
git switch main
```

切回后，文件恢复为 `main` 分支保存的版本，输出语句仍为 `printf("hello ICS");`。将其修改为：

```
printf("hello from main");
```

保存并提交：

```
git diff -- main.c
git add main.c
git commit -m "Change greeting on main branch"
```

生成提交 `604d651`。此时两个分支都从共同版本 `c3e52ef` 出发，将同一行代码改成了不同内容。

3. 合并 feature 分支并观察冲突

在 `main` 分支执行：

```
git merge feature
git status
```

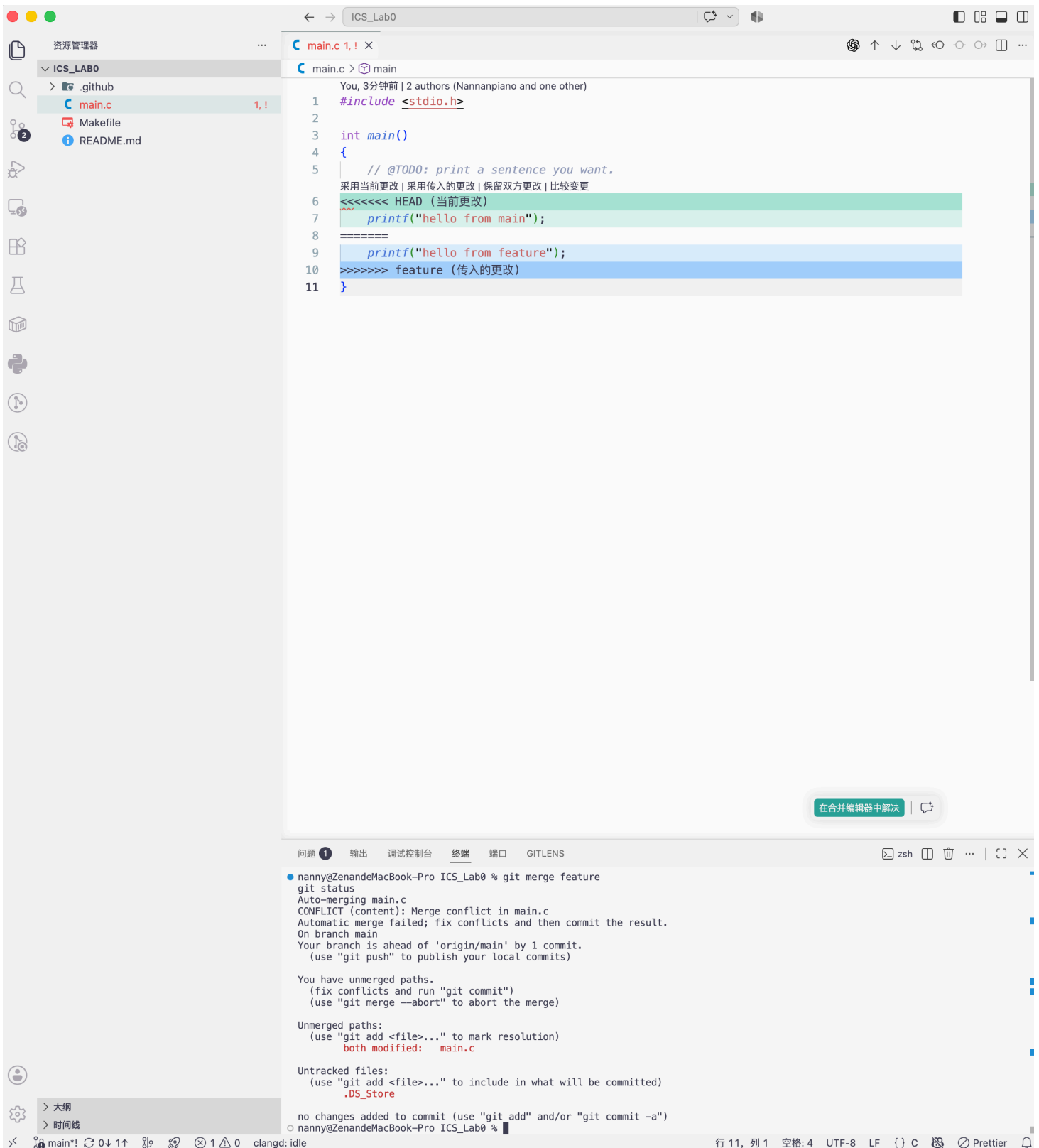
终端显示：

```
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

`git status` 显示存在未合并路径，并将 `main.c` 标记为 `both modified`。编辑器中的冲突区域为：

```
<<<<<<< HEAD
    printf("hello from main");
=====
    printf("hello from feature");
>>>>>> feature
```

HEAD 一侧是当前 `main` 分支的内容，`feature` 一侧是准备合入的内容。由于两个分支相对于共同祖先对同一行进行了不同修改，Git 无法自动决定最终代码，因此需要手动处理。



4. 手动解决冲突并完成合并

我选择保留两个分支的输出，删除 <<<<<< HEAD 、 ===== 、 >>>>>> feature 三个冲突标记。最终的 main.c 为：

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("hello from main");
    printf("hello from feature");
}
```

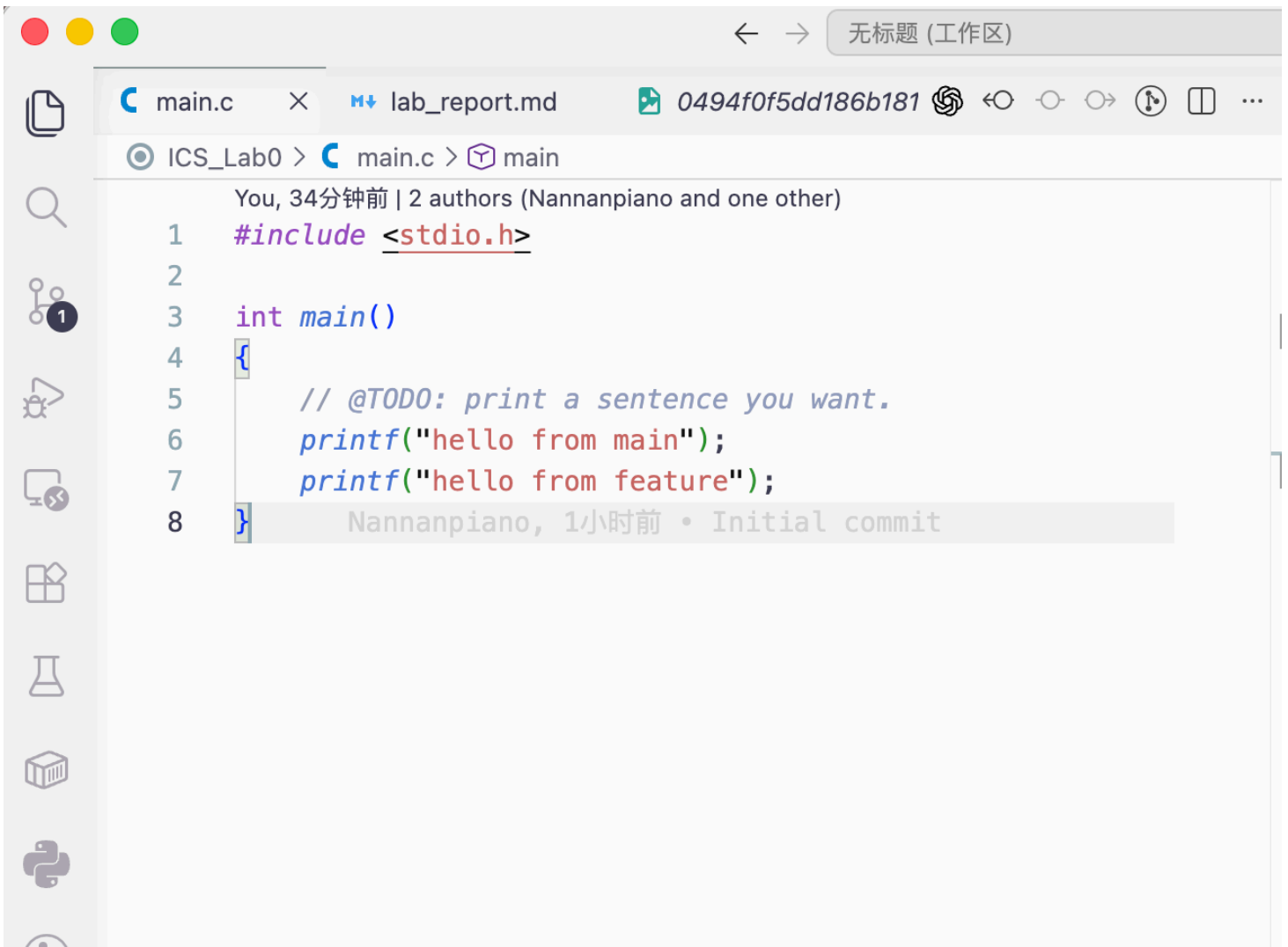
保存后执行：

```
git add main.c
git commit -m "Merge feature into main and resolve greeting conflict"
git status
git log --oneline --graph --all -6
```

`git add main.c` 将手动处理后的文件放入暂存区，标记该文件的冲突已解决；随后 `git commit` 完成本次合并，生成合并提交 `486dfaf`。提交图的核心结构如下：

```
* 486dfaf Merge feature into main and resolve greeting conflict
|\
| * f9dc407 Change greeting on feature branch
* | 604d651 Change greeting on main branch
|/
* c3e52ef Complete main.c TODO
* adedbb9 Initial commit
```

提交图显示，两条分支分别产生修改后，通过 `486dfaf` 汇合。此时状态中不再有未合并路径，说明冲突处理和合并提交已经完成。



```
nanny@ZenandeMacBook-Pro ICS_Lab0 % git add main.c
git commit -m "Merge feature into main and resolve greeting conflict"
git status
git log --oneline --graph --all -6
[main 486dfaf] Merge feature into main and resolve greeting conflict
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)

Untracked files:
(use "git add <file>..." to include in what will be committed)
.DS_Store

nothing added to commit but untracked files present (use "git add" to track)
* 486dfaf (HEAD -> main) Merge feature into main and resolve greeting conflict
| \
| * f9dc407 (feature) Change greeting on feature branch
* | 604d651 Change greeting on main branch
| /
* c3e52ef (origin/main, origin/HEAD) Complete main.c TODO
* adedbb9 Initial commit
```

5. 推送合并结果

执行：

```
git push  
git status
```

实际推送输出包含：

```
To https://github.com/nannanpiano/ICS_Lab0.git  
c3e52ef..486dfaf  main -> main
```

随后状态显示：

```
On branch main  
Your branch is up to date with 'origin/main'.
```

这说明合并结果已推送，且本地 `main` 与远程跟踪分支 `origin/main` 一致。状态中仍有未跟踪的 `.DS_Store`，它没有加入任何实验提交，也不影响合并结果。此次推送上传了 `main` 中包含的双方提交历史，并未单独创建远程 `feature` 分支。